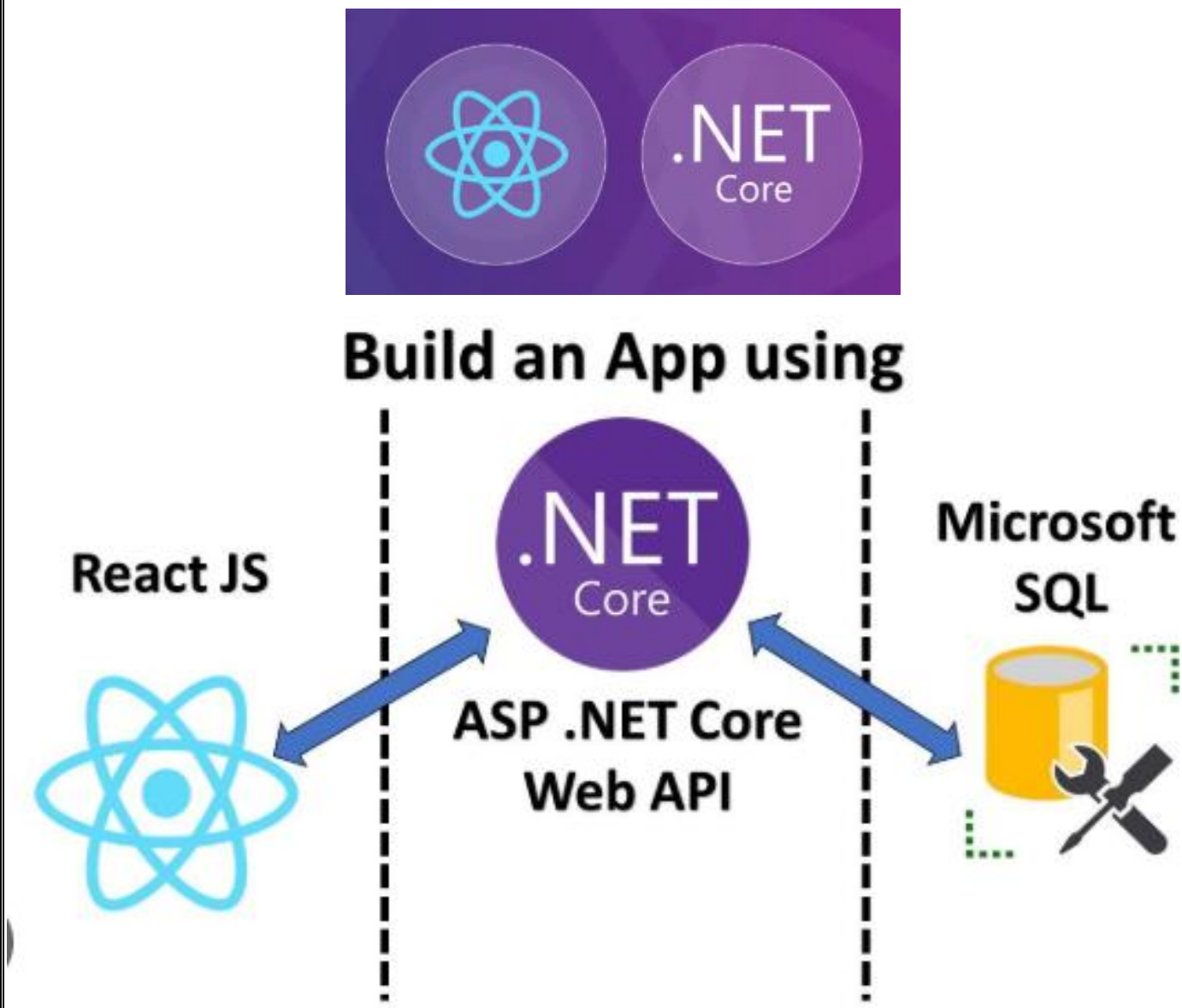

100+ Dot Net + React

Interview Questions and Answers

100+ Dot Net + React Interview Q&A

Dot Net 0–2 Years Experience (Junior Developer)

1. What is the difference between value types and reference types in C#?

Value types store data directly in memory (stack), while reference types store a reference to the memory location (heap). Value types include int, float, struct; reference types include class, string, array. Changes to value types don't affect the original variable, unlike reference types.

```
int a = 5; int b = a; b = 10; // a still 5
class MyClass { public int x = 1; }
MyClass obj1 = new MyClass(); MyClass obj2 = obj1; obj2.x = 5; // obj1.x = 5
```

2. Explain boxing and unboxing with an example.

Boxing is converting a value type to object type; unboxing extracts the value type from the object. Boxing stores value in heap, unboxing retrieves it to stack. It's useful when working with collections.

```
int num = 123;
object obj = num; // Boxing
int result = (int)obj; // Unboxing
Console.WriteLine(result);
```

3. What are access modifiers in C#?

Access modifiers define the scope and visibility of class members. Common types are public, private, protected, internal. They control how members are accessed inside or outside the class.

```
public class MyClass { private int x = 5; protected void Show() { } }
```

4. What is the difference between const, readonly, and static?

const is compile-time constant, readonly is runtime constant, static means shared across instances. const must be initialized at declaration, readonly can be in constructor.

```
const int a = 10;
readonly int b; public MyClass() { b = 20; }
static int count = 0;
```

5. How is var different from dynamic in C#?

var is statically typed and resolved at compile time; dynamic is resolved at runtime. var needs initialization at declaration, dynamic can change type at runtime.

```
var x = "Hello"; // string
dynamic y = 10; y = "World"; // valid
Console.WriteLine(y);
```

6. What are the 4 pillars of OOP? Provide code examples.

The four pillars are Encapsulation, Abstraction, Inheritance, and Polymorphism. These principles help build modular and reusable code.

```
// Encapsulation
class Person { private string name; public void SetName(string n) { name = n; } }
// Inheritance
class Animal { public void Eat() { } } class Dog : Animal { }
// Polymorphism
class Shape { public virtual void Draw() {} } class Circle : Shape { public override void Draw() {} }
```


Sai Reddy
saireddy-dotnetfs

```
// Abstraction
abstract class Vehicle { public abstract void Drive(); } class Car : Vehicle { public override void Drive() {
} }
```

7. What is the difference between abstraction and encapsulation?

Abstraction hides complexity using abstract/interface, while encapsulation hides data using access modifiers. Abstraction focuses on what, encapsulation on how.

```
// Abstraction
abstract class Vehicle { public abstract void Drive(); }
// Encapsulation
class BankAccount { private int balance; public void Deposit(int amt) { balance += amt; } }
```

8. What is method overloading vs method overriding?

Overloading allows multiple methods with same name and different parameters in same class. Overriding allows changing base class method behavior using override in derived class.

```
// Overloading
void Print(int a) {} void Print(string s) {}
// Overriding
class A { public virtual void Show() {} } class B : A { public override void Show() {} }
```

9. What is the difference between ADO.NET and EF?

ADO.NET is low-level and manual with SQL commands, EF is high-level ORM that uses LINQ and entities. EF reduces boilerplate and handles mapping.

```
// ADO.NET
SqlCommand cmd = new SqlCommand("SELECT * FROM Users", conn);
// EF
var users = context.Users.ToList();
```

10. What are DbContext and DbSet in EF?

DbContext manages database connection and change tracking. DbSet represents a table and allows CRUD operations. Together they enable ORM.

```
public class MyContext : DbContext { public DbSet<User> Users { get; set; } }
var user = context.Users.First();
```

11. How do you fetch data using LINQ?

LINQ allows querying collections and databases using query or method syntax. It works with EF for querying tables.

```
var result = from u in context.Users where u.Age > 20 select u;
var list = context.Users.Where(u => u.Age > 20).ToList();
```

12. What is the MVC architecture?

MVC (Model-View-Controller) separates application into three components. Model handles data, View handles UI, Controller handles logic and input.

```
public class HomeController : Controller {
  public ActionResult Index() { return View(); }
}
```

13. Difference between ViewBag, ViewData, and TempData?

ViewBag is dynamic, ViewData is dictionary, both are for passing data to view. TempData persists across redirects.

ViewBag.Name = "John"; ViewData["Age"] = 25; TempData["Msg"] = "Saved";

14. How do you call an API from a controller?

Use HttpClient to make HTTP requests from a controller to an API. Handle async calls with await.

```
HttpClient client = new HttpClient();
var res = await client.GetAsync("https://api.com/data");
string data = await res.Content.ReadAsStringAsync();
```

15. What is Razor syntax?

Razor is a markup syntax for embedding C# code in .cshtml views. It starts with @ symbol. It's used for generating HTML dynamically.

```
<p>Hello @Model.Name</p>
@if(Model.IsActive) { <p>Active</p>; }
```

16. How do you use JavaScript or jQuery in .NET MVC?

You can include JS/jQuery in views using <script> tags or layout. Use jQuery to manipulate DOM or call APIs.

```
<script src="~/Scripts/jquery-3.6.0.min.js"></script>
<script>
  $(document).ready(function() { alert("Ready"); });
</script>
```

Dot Net 3–6 Years Experience (Mid-Level Developer)

1. What are delegates and events? Give real-time examples.

Delegates are type-safe function pointers used to call methods indirectly. Events are wrappers around delegates to follow the publisher-subscriber model, like button click handlers.

```
public delegate void Notify();
public event Notify OnCompleted;
void Print() { Console.WriteLine("Done"); }
OnCompleted += Print; OnCompleted();
```

2. What is the difference between IEnumerable and IQueryable?

IEnumerable executes queries in memory and supports LINQ to Objects. IQueryable executes queries in the database and supports LINQ to Entities. IQueryable is more efficient for DB queries.

```
IEnumerable<User> users = context.Users.ToList();
IQueryable<User> query = context.Users.Where(u => u.Age > 30);
```

3. Explain async/await and Task.

async/await is used for asynchronous programming to prevent blocking UI or threads. Task represents ongoing operations. It helps improve responsiveness.

```
public async Task<string> GetData() {
  HttpClient client = new HttpClient();
  var res = await client.GetStringAsync("https://api.com");
  return res;
}
```

4. What is Code First vs Database First in EF?

Code First creates DB from model classes; Database First creates model from existing DB. Code First uses migrations, Database First uses .edmx designer.

```
// Code First
public class Student { public int Id; public string Name; }
// Database First
// EDMX file used to generate models from DB
```

5. What are migrations in EF Core?

Migrations track and apply schema changes in the database. Use Add-Migration to create, Update-Database to apply. It's part of Code First development.

```
Add-Migration AddStudentsTable
Update-Database
```

6. How do you handle lazy loading vs eager loading?

Lazy loading loads related data when accessed; eager loading loads related data with the main query using .Include(). Eager is better for performance in most cases.

```
// Eager loading
var students = context.Students.Include(s => s.Courses).ToList();
// Lazy loading (requires config)
var student = context.Students.First(); var courses = student.Courses;
```

7. How do you secure Web APIs using JWT?

JWT (JSON Web Token) is used for stateless authentication. The server issues a token after login and the client sends it in the header for each request.

```
services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
.AddJwtBearer(options => { options.TokenValidationParameters = new TokenValidationParameters {
/*config*/ }; });
[Authorize] public IActionResult GetData() => Ok("Secure");
```

8. What is middleware in ASP.NET Core?

Middleware are software components in the HTTP pipeline that handle requests/responses. Common examples: authentication, logging, CORS.

```
app.UseRouting();
app.UseAuthentication();
app.UseAuthorization();
app.UseEndpoints(endpoints => { endpoints.MapControllers(); });
```

9. Difference between .NET Framework, .NET Core, and .NET 5/6/7?

.NET Framework is Windows-only and older; .NET Core is cross-platform and modular; .NET 5+ is unified platform for all workloads, replacing .NET Framework and Core.

```
.NET Framework - Windows apps
.NET Core - Cross-platform
.NET 5/6/7 - Unified, future-ready
```

10. What is the Singleton pattern? Provide C# code.

Singleton ensures only one instance of a class exists globally. It's useful for config, logging, caching.

```
public class Logger {
  private static Logger instance;
  private Logger() {}
  public static Logger Instance => instance ??= new Logger();
}
```

11. Explain Dependency Injection and its benefits.

DI injects dependencies into a class instead of creating them. It improves testability, decouples code, and follows SOLID principles.

```
public interface IService { void Run(); }
public class MyService : IService { public void Run() {} }
public class Controller { private IService _svc; public Controller(IService svc) { _svc = svc; } }
```

12. What is unit testing? How do you use xUnit/NUnit?

Unit testing tests small, independent parts of code. xUnit/NUnit are popular frameworks. Tests are methods with assertions.

```
[Fact] public void Add_ReturnsSum() {
  var calc = new Calculator();
  Assert.Equal(5, calc.Add(2, 3));
}
```

13. How do you deploy .NET applications using Azure DevOps?

Use pipelines to build and release apps. Define steps in YAML or classic editor. Use tasks like DotNetCoreCLI, PublishBuildArtifacts, AzureWebApp.

```
trigger: [main]
jobs:
- job: Build
  steps:
  - task: DotNetCoreCLI@2
    inputs: { command: 'build', projects: '**/*.csproj' }
```

Dot Net 7–14 Years Experience (Senior Developer / Team Lead)

1. What is a layered vs a microservices architecture?

Layered architecture separates concerns into layers like UI, BLL, DAL, and DB. Microservices architecture breaks app into small, independent services that communicate via APIs.

Layered: UI → BLL → DAL → DB

Microservices: AuthService, OrderService, PaymentService (all separate APIs)

2. Explain Domain-Driven Design (DDD) principles.

DDD emphasizes modeling real-world domains via Entities, Value Objects, Aggregates, and Bounded Contexts. It aligns code structure with business logic.

```
class Order { public OrderId Id; public List<OrderItem> Items; }
value struct OrderId { Guid Id; }
```

3. What are SOLID principles? Provide examples.

SOLID includes Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, Dependency Inversion — promotes clean and maintainable code.

```
// SRP
class InvoicePrinter { void Print(Invoice inv) {} }
// OCP
abstract class Shape { public abstract double Area(); }
class Circle : Shape { public override double Area() => 3.14 * r * r; }
```

4. How do you handle logging and exception management in .NET Core?

Use built-in ILogger, try-catch, and middleware for global exception handling. Log levels help track app behavior.

```
public void Configure(IApplicationBuilder app) {
  app.UseExceptionHandler("/Home/Error");
}
_logger.LogError(ex, "An error occurred");
```

5. How do you improve the performance of a .NET application?

Use caching, async methods, reduce DB calls, enable compression, and profiling tools. Optimize EF queries and avoid unnecessary object allocations.

```
var data = await memoryCache.GetOrCreateAsync("key", e => db.GetDataAsync());
```

6. What are best practices for securing .NET APIs?

Use HTTPS, JWT authentication, role-based access, input validation, rate limiting, CORS policy, and secure headers. Avoid exposing sensitive info.

```
[Authorize(Roles = "Admin")]
[HttpPost] public IActionResult Create([FromBody] User user) { }
```

7. What is rate limiting and how is it implemented?

Rate limiting restricts the number of API requests in a time frame to prevent abuse. Use libraries like AspNetCoreRateLimit.

```
services.AddMemoryCache();
services.Configure<IpRateLimitOptions>(config);
app.UseIpRateLimiting();
```

8. How do you deploy .NET applications to Azure?

Use Azure DevOps or GitHub Actions to build and deploy to Azure App Service, VM, or Container. Publish profile or pipelines automate deployment.

```
- task: AzureWebApp@1
  inputs: { appName: 'MyApp', package: '$(System.DefaultWorkingDirectory)/**/*.zip' }
```

9. What are Azure Functions and how do you use them?

Azure Functions are serverless components triggered by events like HTTP, timer, or queue. Use them for background jobs, lightweight APIs.

```
[FunctionName("HelloFunction")]
public IActionResult Run([HttpTrigger] HttpRequest req) => new OkObjectResult("Hello");
```

10. What is the difference between App Service and Container Instances?

App Service is PaaS for web apps with built-in scaling and deployment. Container Instances run Docker containers with full control but minimal orchestration.

App Service: Easy deployment for web APIs

Container Instance: Lightweight, isolated containers

11. How do you mentor junior developers?

Provide code reviews, explain best practices, assign growth tasks, pair program, and encourage questions. Focus on clean code and problem-solving.

Set goals → Guide via feedback → Encourage learning

12. Describe your involvement in Agile/Scrum ceremonies.

Participate in sprint planning, standups, reviews, and retros. Help refine backlog and break tasks. Collaborate with team to meet sprint goals.

Daily Standup → Sprint Planning → Review → Retrospective

13. How do you manage technical debt in a large .NET project?

Track in backlog, prioritize refactoring, automate tests, follow coding standards, and schedule tech debt sprints. Communicate trade-offs clearly.

Create tickets → Refactor iteratively → Monitor via SonarQube

React JS 0–2 Years Experience (Junior React Developer)

1. What is React? Why use it?

React is a JavaScript library for building user interfaces. It helps build reusable UI components and efficiently updates the DOM using a virtual DOM.

```
const App = () => <h1>Hello React</h1>;
```

2. What is JSX?

JSX is a syntax extension for JavaScript that looks like HTML and is used in React to describe the UI structure.

```
const element = <h1>Welcome!</h1>;
```

3. What are props and state in React?

Props are read-only data passed to components, while state is local data managed within a component that can change.

```
function Hello({ name }) { return <p>Hello {name}</p>; }
const [count, setCount] = useState(0);
```

4. Explain the lifecycle of a React component.

For class components: constructor → render → componentDidMount → updates via componentDidUpdate → cleanup via componentWillUnmount.

```
componentDidMount() { console.log('Mounted'); }
```

5. What are functional vs class components?

Functional components are simpler and use hooks; class components use lifecycle methods and this.state.

```
const Greet = () => <h1>Hello</h1>;
class Greet extends React.Component { render() { return <h1>Hello</h1>; } }
```

6. How do you handle events in React?

Use camelCase event names and pass event handler functions.

```
<button onClick={() => alert('Clicked!')}>Click</button>
```

7. How do you update state in a React component?

Use setState in class components and useState setter in functional components.

```
const [count, setCount] = useState(0);
setCount(count + 1);
```

8. What is useState? Show an example.

useState is a React hook to manage state in functional components.

```
const [text, setText] = useState('');
<input value={text} onChange={e => setText(e.target.value)} />
```

9. How do you render a list in React?

Use map() to iterate and render list items, providing a unique key prop.

```
const items = ['A', 'B'];
<ul>{items.map((item, i) => <li key={i}>{item}</li>)}</ul>
```

10. How do you handle form inputs in React?

Use useState to store form values and update on onChange event.

```
const [email, setEmail] = useState('');
<input value={email} onChange={e => setEmail(e.target.value)} />
```

11. What is the purpose of useEffect?

useEffect handles side effects like API calls, subscriptions, or DOM manipulation after rendering.

```
useEffect(() => { console.log('Component mounted'); }, []);
```

12. What is useRef used for?

useRef gives a reference to DOM elements or stores mutable values without re-rendering.

```
const inputRef = useRef();
<input ref={inputRef} />;
<button onClick={() => inputRef.current.focus()}>Focus</button>
```

React JS 3–6 Years Experience (Mid-Level React Developer)

1. How does useMemo help performance?

useMemo caches expensive calculations and returns memoized values unless dependencies change. It avoids recalculating on every render.

```
const total = useMemo(() => computeTotal(cart), [cart]);
```

2. What's the difference between useCallback and useMemo?

useCallback memoizes functions; useMemo memoizes computed values. Use useCallback to prevent unnecessary re-creations of handlers.

```
const handleClick = useCallback(() => doSomething(id), [id]);
```

3. What are custom hooks? When do you create one?

Custom hooks reuse logic across components. Create one when logic is shared (e.g., form handling, fetch).

```
function useCounter() { const [c, setC] = useState(0); return [c, () => setC(c+1)]; }
```

4. What are controlled vs uncontrolled components?

Controlled components use state to manage input value; uncontrolled use refs. Controlled offers better control over form logic.

```
<input value={name} onChange={e => setName(e.target.value)} /> // Controlled
<input ref={inputRef} /> // Uncontrolled
```

5. How do you implement conditional rendering?

Use if, ternary operator, or logical && to render conditionally.

```
{isLoggedIn ? <Dashboard /> : <Login />}
```

6. How do you share state between components?

Lift state up to a common parent or use Context API or global state libraries like Redux.

```
const [data, setData] = useState();
<Child data={data} setData={setData} />
```

7. How do you use the Context API?

Create context, wrap components in provider, and consume using useContext.

const ThemeContext = createContext();

<ThemeContext.Provider value="dark"><Child /></ThemeContext.Provider>

const theme = useContext(ThemeContext);

8. Compare Redux with Context API.

Context is built-in and good for small apps. Redux is better for complex apps with middleware, dev tools, and large state trees.

Redux = predictable + scalable, Context = simple sharing

9. What is Redux middleware? Examples?

Middleware intercepts actions between dispatch and reducers. Used for logging, async ops (e.g., thunk, saga).

const logger = store => next => action => { console.log(action); return next(action); };

10. How do you implement routing in React?

Use react-router-dom. Wrap app with BrowserRouter, use Routes and Route.

<BrowserRouter><Routes><Route path="/" element={<Home />} /></Routes></BrowserRouter>

11. Difference between BrowserRouter and HashRouter?

BrowserRouter uses clean URLs and HTML5 history. HashRouter uses # and is better for older server setups.

<BrowserRouter> vs <HashRouter> // path vs hash-based navigation

12. What causes re-rendering and how do you prevent it?

Re-renders happen on state/prop change. Prevent using React.memo, useCallback, useMemo, and proper key usage.

const MemoComponent = React.memo(MyComponent);

13. What is React.memo?

React.memo prevents re-render if props haven't changed. Good for optimizing functional components.

const MyComponent = React.memo(({ name }) => {name});

React JS 7–14 Years Experience (Senior React Developer / Lead)

1. How do you structure a large-scale React application?

Use feature-based folders, separate logic and UI, use reusable components, centralize state, and add testing.

```
src/
  ├── features/Users
  ├── components/common
  ├── hooks/
  ├── services/api.js
  └── App.jsx
```

2. What is a Higher Order Component (HOC)?

HOC is a function that takes a component and returns a new one with enhanced behavior.

const withLogger = Component => props => { console.log(props); return <Component {...props} />; };

3. Explain Render Props with an example.

Render Props share code by passing a function as a child.

<Mouse render={({x, y}) => <h1>Mouse at {x},{y}</h1>} />

4. What are compound components?

Compound components share implicit state through context and are used together.

<Tabs><Tabs.Tab>One</Tabs.Tab><Tabs.Panel>Panel</Tabs.Panel></Tabs>

5. What are the pros/cons of Redux Toolkit?

Pros: Less boilerplate, built-in createSlice, immer. Cons: Abstracts internal Redux behavior, requires Redux knowledge.

const counterSlice = createSlice({ name: 'counter', initialState: 0, reducers: { increment: state => state + 1 } });

6. Compare Redux Saga vs Redux Thunk.

Thunk is simpler, uses functions; Saga is more powerful for complex async flows using generator functions.

// Thunk

dispatch(async dispatch => { const data = await api(); dispatch(setData(data)); });

// Saga

function* fetchData() { const data = yield call(api); yield put(setData(data)); }

7. How do you manage side effects in Redux?

Use middleware like Redux Thunk or Redux Saga to handle async API calls, delays, or other effects.

const fetchData = () => async dispatch => { const res = await fetch(); dispatch(setData(res)); };

8. How do you lazy-load components in React?

Use React.lazy() with Suspense for dynamic import.

const LazyComp = React.lazy(() => import('./MyComponent'));

<Suspense fallback={<div>Loading...</div>}><LazyComp /></Suspense>

9. What is code splitting? How does React support it?

Code splitting loads only necessary code. React supports it with React.lazy() and dynamic imports.

const Page = React.lazy(() => import('./Page'));

10. How do you write unit tests in React using Jest and React Testing Library?

Use render() to mount the component and assertions to check behavior.

render(<Greeting name="John" />); expect(screen.getByText(/John/)).toBeInTheDocument();

11. How do you prevent XSS in React?

React escapes content by default. Avoid dangerouslySetInnerHTML unless sanitized.

{userInput} // Safe

12. What are common security risks in React apps?

XSS, CSRF, insecure storage, exposed API keys. Sanitize input, use HTTPS, env vars, auth tokens. Never store JWT in localStorage, prefer HttpOnly cookies

13. What is Webpack? How does it work with React?

Webpack bundles JS, CSS, assets. It compiles JSX, processes loaders, and outputs optimized builds.

module.exports = { entry: './src/index.js', module: { rules: [{ test: /\.jsx$/, loader: 'babel-loader' }] } };

14. What are source maps and why are they important?

Source maps map minified code to original source, helping debug in browser dev tools.

devtool: 'source-map' in Webpack shows original code in browser

Sai Reddy
saireddy-dotnetfs

React JS 15–20 Years Experience (Architect / Principal Engineer)

1. How do you design a microfrontend architecture with React?

Split the UI into independent deployable modules using Module Federation or iframe strategies. Use a shell app to load remotes.

```
// webpack.config.js
module.exports = { plugins: [new ModuleFederationPlugin({ name: 'app1', exposes: {'./Button': './src/Button'} })] };
```

2. How do you build a scalable component library?

Use Storybook for isolated dev, TypeScript for types, enforce design tokens, version via Lerna or Nx.

```
export const Button = ({label}) => <button className="btn">{label}</button>;
```

3. How would you migrate a legacy JavaScript/Angular app to React?

Use a hybrid approach via Web Components or iframe, migrate route-by-route, start with shared services.

```
// Wrap React in custom element and load in Angular
customElements.define('my-react', reactToWebComponent(App, React, ReactDOM));
```

4. How do you manage performance at scale in React apps?

Use code-splitting, memoization, virtual lists, avoid prop drilling, audit using Lighthouse/React Profiler.

```
import { FixedSizeList as List } from 'react-window'; // for large list virtualization
```

5. What's your strategy for code-splitting across large teams/projects?

Split per route/component using React.lazy, use Module Federation for multi-team delivery, async load dependencies.

```
const AdminPanel = React.lazy(() => import('adminApp/AdminPanel'));
```

6. How do you deploy React apps with CI/CD (e.g., GitHub Actions, Azure DevOps)?

Use GitHub Actions to lint, test, build, and deploy to Azure/Netlify. Use cache and secrets for security.

```
steps:
- uses: actions/setup-node
- run: npm ci && npm run build
- uses: azure/webapps-deploy@v2
```

7. How do you manage environment variables securely in React?

Store in .env, prefix with REACT_APP_, and avoid exposing secrets; inject via CI/CD securely.

```
REACT_APP_API_URL=https://api.domain.com
```

8. How do you mentor React developers on best practices?

Set coding standards, encourage design patterns, pair program, review code, and guide via architecture decisions.

Use ESLint, Prettier, TypeScript, and Storybook to enforce best practices

9. How do you evaluate and introduce new libraries or tools to a React codebase?

Check community support, maintenance, performance, fit to needs, and try in pilot feature before adoption.

Create a POC branch, test, review DX and bundle impact, then discuss with team

10. How do you collaborate with backend teams in an API-driven React app?

Use OpenAPI/Swagger contracts, versioned APIs, shared DTOs/types, and plan with backend in refinement sessions.

```
fetch('/api/v1/users').then(res => res.json()).then(setUsers);
```